# PRACTICAL - 12

Aim: Word frequency (WordCounter.java + Driver.java)

## Code Screenshot:

Visual Studio Code - WordCounter.java

WordCounter.java

```java
import java.util.*;

public class WordCounter {
    public static Map<String, Integer> count(String text) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : text.toLowerCase().split("[^a-z0-9]+")) {
            if (!word.isEmpty()) counts.put(word, counts.getOrDefault(word, 0) + 1);
        }
        return counts;
    }

    public static List<Map.Entry<String, Integer>> sorted(Map<String, Integer> counts) {
        List<Map.Entry<String, Integer>> result = new ArrayList<>(counts.entrySet());
        result.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()));
        return result;
    }
}
```

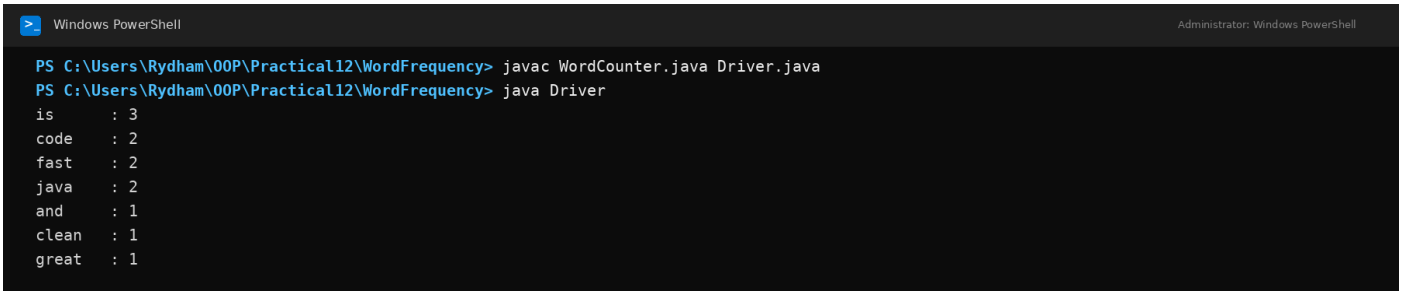
Visual Studio Code - Driver.java

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        String text = "Java is great and Java is fast. Fast code is clean code!";
        for (var entry : WordCounter.sorted(WordCounter.count(text))) {
            System.out.printf("%-7s : %d%n", entry.getKey(), entry.getValue());
        }
    }
}
```

## Output Screenshot:




## Learning Outcome:

- Mastered key-value associative data storage using HashMap.
- Used map.getOrDefault() for concise frequency counter accumulation.
- Sorted map entry sets dynamically using Comparator chaining.
- Tokenized and sanitized unstructured text with regular expressions.
- Understood hash table average O(1) retrieval performance.

# PRACTICAL - 12

Aim: Unique visitors (VisitorTracker.java + Driver.java)

## Code Screenshot:

Visual Studio Code - VisitorTracker.java

VisitorTracker.java

```java
import java.util.*;

public class VisitorTracker {
    private final Set<String> unique = new HashSet<>();
    private final Set<String> ordered = new LinkedHashSet<>();

    public void record(String name) {
        unique.add(name);
        ordered.add(name);
    }

    public int uniqueCount() { return unique.size(); }

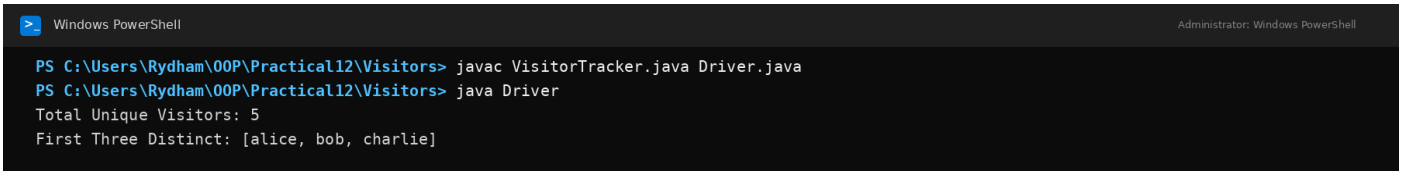
    public List<String> firstThree() {
        return ordered.stream().limit(3).toList();
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        VisitorTracker tracker = new VisitorTracker();
        String[] visits = {"alice", "bob", "alice", "charlie", "bob", "david", "emma"};
        for (String name : visits) tracker.record(name);
        System.out.println("Total Unique Visitors: " + tracker.uniqueCount());
        System.out.println("First Three Distinct: " + tracker.firstThree());
    }
}
```

## Output Screenshot:




## Learning Outcome:

- Compared unordered HashSet against insertion-ordered LinkedHashSet.
- Implemented automatic data deduplication using Set semantics.
- Preserved arrival order while filtering duplicate telemetry events.
- Streamed collection subsets cleanly with stream().limit().
- Evaluated memory and time trade-offs across distinct Java collection types.

# PRACTICAL - 12

Aim: Leaderboard (For Advanced Learners) (Leaderboard.java + Driver.java)

## Code Screenshot:

Visual Studio Code - Leaderboard.java

Leaderboard.java

```java
import java.util.*;

public class Leaderboard {
    public static List<Map.Entry<String, Integer>> topN(Map<String, Integer> scores, int n) {
        List<Map.Entry<String, Integer>> ranked = new ArrayList<>(scores.entrySet());
        ranked.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()));
        return ranked.subList(0, Math.min(n, ranked.size()));
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        Map<String, Integer> scores = Map.of(
                "Aarav", 850, "Diya", 920, "Kabir", 920,
                "Meera", 780, "Riya", 850);
        int rank = 1;
        for (var player : Leaderboard.topN(scores, 3)) {
            System.out.println(rank++ + ". " + player.getKey() + " - " + player.getValue());
        }
    }
}
```

## Output Screenshot:




## Learning Outcome:

- Stored player scores in a Map using player names as keys.
- Ranked players by descending score with a Comparator.
- Applied alphabetical ordering to resolve equal-score ties.
- Selected the top N players from the complete score map.
- Displayed leaderboard positions with their corresponding scores.